

```

import numpy as np
import matplotlib.pyplot as plt
import json
import sys
import matplotlib.pyplot
import scipy.linalg

import plotly
import plotly.graph_objs as go

```

Introduction to numerical methods

János Török

Class 2: Linear algebra

Permutations

Enumerate all permutations of the number 1,2,3

```

l = [ 1, 2, 3] # the list of numbers to be permuted
permutations = []
for i1 in range(3):
    for i2 in range(3):
        if i1 == i2: continue
        for i3 in range(3):
            if i1 == i3 or i2 == i3: continue
            permutations.append([ l[i1], l[i2], l[i3] ])

permutations

```

It would be silly to write a different function for different number of elements. Let us write a general code.

- Recursive. Take all elements successively and add all permutations of the rest in all cases
- If the function receives a single element, we are done, return it.
- The return value of the function is a list of permuted lists, so in the previous case we should return the list of list of that last element

```

def rpermutations(l):
    print("Kukucs",l)
    # if we have one element return the list of list of it
    if len(l) == 1:
        print("I have only one element: ", l[0])
        return [l]

```

```

# iterate through all elements
permutations = []
for i in range(len(l)):
    ltmp = l.copy()
    my = ltmp.pop(i) # remove the element from the list and
    permute the rest
    for r in rpermutations(ltmp):
        # we have to add all combinations
        permutations.append([my] + r)
print("I am returning:", permutations)
return permutations

l = [1, 2, 3, 4]
p = rpermutations(l)
print(p)

```

We define the Levi-Civita symbol as

$$\varepsilon_{a_1 a_2 a_3 \dots a_n} = \begin{cases} +1 & \text{if } (a_1, a_2, a_3, \dots, a_n) \text{ is an even permutation of } (1, 2, 3, \dots, n) \\ -1 & \text{if } (a_1, a_2, a_3, \dots, a_n) \text{ is an odd permutation of } (1, 2, 3, \dots, n) \\ 0 & \text{otherwise} \end{cases}$$

The permutation is when we change two indices e.g. $(1, 2, 3, 4) \rightarrow (1, 3, 2, 4)$.

Let us create a function which calculates the number of swaps there are between two permutations:

- for each position i we search the element and swap it to its place

```

def swap(l, i1, i2):
    tmp = l[i1]
    l[i1] = l[i2]
    l[i2] = tmp

def numberofswap(l1, l2):
    # we will modify l2, so do it on a copy
    ll2 = l2.copy()
    noswap = 0
    if len(l1) != len(ll2): # Just to be safe
        return "Error: the length of the arrays must be equal"
    for i in range(len(l1)):
        v = l1[i]
        try:
            j = ll2.index(v)
        except:
            return "Error: the lists differ in elements"
        if i != j:
            swap(ll2, i, j)
            noswap += 1
    return noswap

```

Now we create an 3 dimensional Levi-Civita symbol

```
E = np.zeros((3,3,3),dtype=int)
for p in rpermutations([0,1,2]):
    ns = numberofswap([0,1,2],p)
    print(ns)
    if ns%2 == 0:
        E[p[0],p[1],p[2]] = 1
    else:
        E[p[0],p[1],p[2]] = -1
```

E

Vector space

Here we consider only n -dimensional vector spaces which is defined by its bases and we will work with the vectors defined by this bases. During the lecture we mainly work with real numbers but numpy does not care and everything should work with complex ones.

Vectors: Vectors $v \in V$ can be represented by simple numpy array or an $1 \times n$ matrix, people often use one dimensional arrays for vectors, and for most purposes it is fine, but not everything works with it

```
a = np.zeros(3,dtype=float) # one dimensional array
b = np.ones((3,1),dtype=float) # column vector
c = np.ones((1,3),dtype=float) # row vector
print(a)
print(b)
print(c)
```

The most basic operation we have in a vector space is the addition and multiplication by a number, The transpose function mirrors the object to the diagonal

```
print(c*3.7)
print(c+c)
print(b+c.T)
```

For sake of exercise let us try complex numbers

```
z = np.ones(3,dtype=complex)
print(z)

#As you can see the complex unit is denoted by j
z[1] = 5+3.6j
print(z,z.real,z.imag)
```

Scalar product

$$s = (a, b) = \sum_{i=1}^n a_i b_i$$

```
#elementwise multiplication:  
a = np.arange(3)  
b = np.arange(1,4)  
  
print(a,b,a * b, (a*b).sum())
```

`numpy.dot(a,b)`:

- If both `a` and `b` are 1-D arrays, it is inner product of vectors (without complex conjugation).
- If both `a` and `b` are 2-D arrays, it is matrix multiplication, but using `matmul` or `a @ b` is preferred.
- If either `a` or `b` is 0-D (scalar), it is equivalent to multiply and using `numpy.multiply(a, b)` or `a * b` is preferred.

However `matmul` is slower for matrix sizes ~ 30 or smaller, so feel free to use `numpy.dot`!

```
print(np.dot(a,b))  
#other uses but I will mostly stick with the previous one:  
print(a.dot(b),b.dot(a))
```

Length of a vector

$$||a|| = \sqrt{(a,a)}$$

```
#however `numpy.linalg.norm` is preferred  
print(np.sqrt(np.dot(a,a)), np.linalg.norm(a))
```

Matrix

Let A and $m \times n$ matrix, then it read as

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Product of a matrix A and a vector v

$$(Av)_i = \sum_{j=1}^n A_{ij} v_j$$

Matrix product:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{pmatrix}$$

The product will have the following structure:

$$C = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{pmatrix}$$

The product is defined as:

$$(AB)_{ij} = C_{ij} = c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj} = \sum_{k=1}^n a_{ik}b_{kj}$$

```
m = 4
n = 3
p = 2
A = np.random.randint(5, size=(m,n))
v = np.random.randint(5, size=(n,1))
w = v[:,0]
print("A=", A)
print("v=", v)
print("w=", w)
print("Av=", np.dot(A, v), "\n", "Av=", np.matmul(A, v))
print("Aw=", np.dot(A, w), "\n", "Aw=", np.matmul(A, w))
```

Remarks: if w is defined as a vector, then the result will also be a vector

```
B = np.random.randint(5, size=(n,p))
print("B=", B)
print("C=", np.dot(A, B))
print("C=", np.matmul(A, B))

%%timeit
C = np.dot(A, B)

%%timeit
C = np.matmul(A, B)
```

How challenging it is to write a matrix multiplication?

```
def mymm(A, B):
    m, n = A.shape
    nb, p = B.shape
```

```

if n != nb:
    return "Incompatible matrix shapes"
C = np.zeros((m,p),dtype=float)
for mi in range(m):
    for pi in range(p):
        for ni in range(n):
            C[mi,pi] += A[mi,ni] * B[ni,pi]
return C

```

mymm(A,B)

#get rid of one of the summation

```

def mymm2(A,B):
    m,n = A.shape
    nb,p = B.shape
    if n != nb:
        return "Incompatible matrix shapes"
    C = np.zeros((m,p),dtype=float)
    for mi in range(m):
        for pi in range(p):
            C[mi,pi] = (A[mi,:] * B[:,pi]).sum()
    return C

```

mymm2(A,B)

If the matrix A is square say $n \times n$ then it has a determinant:

$$\det(A) = \sum_{i_1, i_2, \dots, i_n} \varepsilon_{i_1 \dots i_n} a_{1, i_1} \dots a_{n, i_n}$$

If $n=2$:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

If $n=3$:

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = aei + bfg + cdh - ceg - bdi - afh.$$

```

n = 3
A = np.random.randint(5, size=(n,n))
print(A)
print(np.linalg.det(A))

```

We can use our permutation to calculate the determinant

```
def mydet(A):
    d = 0.0
    n = len(A)
    for p in rpermutations(list(range(n))):
        d += A[(np.arange(n),p)].prod() * (-
1)**numberofswap(list(range(n)),p)
    return d

mydet(A)
```

Linear operations

Given a vector v in n -dimension all linear operations can be represented by an $n \times m$ matrix if the target space is m dimensional. Here special cases: $n=m=2$

$$u = A v$$

Scaling

All vectors are scaled by a_1 in x and a_2 in y direction:

$A = \begin{pmatrix} a_1 & 0 \\ 0 & a_2 \end{pmatrix}$

```
def scaling_matrix(a1,a2):
    A = np.zeros((2,2),dtype=float)
    A[0,0] = a1
    A[1,1] = a2
    return A
def plotarrow(v1,color,ls='solid'):
    l = np.linalg.norm(v1)
    plt.arrow(*origin, v1[0], v1[1],head_width=l/15, head_length=l/6,
fc=color, ec=color, ls=ls)

v1 = np.array([1,2])
v2 = np.array([-2,-0.7])
A = scaling_matrix(0.7,-0.2)
origin = [0,0]
plt.figure(figsize=(10,7))
plt.xlim(-3,3)
plt.ylim(-3,3)
plotarrow(v1,'r')
plotarrow(np.dot(A,v1),"r",ls="dotted")
plotarrow(v2,'b')
plotarrow(np.dot(A,v2),"b",ls="dotted")

v = np.zeros((6,2),dtype=float)
for i in range(6):
    v[i,0] = np.cos(2*np.pi *(2*i/5+0.05))
    v[i,1] = np.sin(2*np.pi *(2*i/5+0.05))
```

```
plt.plot(v[:,0],v[:,1])
u = np.dot(A,v.T).T
plt.plot(u[:,0],u[:,1],"r-")
```

Projections

if we want to project the vector v to the plane with normal t . Then the projection matrix is

$$P = I - \frac{t t^T}{t^T t}$$

```
def projection(t):
    tt = t / np.linalg.norm(t)
    return np.outer(tt,tt)

#or
#tt = t / np.linalg.norm(t)
#return np.outer(t,t)/(t*t).sum()

t = np.array([1,2])
P = projection(t)
print(P)

#Check whether its determinant is zero
print(np.linalg.det(P))

plt.plot(v[:,0],v[:,1])
u = np.dot(P,v.T).T
plt.plot(u[:,0],u[:,1],"r-")
```

Rotation

If we want to rotate the vector v along the z axis with angle ϕ . Then the rotation matrix is

$$R = \begin{pmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{pmatrix}$$

```
def rotation(phi):
    R = np.zeros((2,2),dtype=float)
    R[0,0] = R[1,1] = np.cos(phi)
    R[0,1] = - np.sin(phi)
    R[1,0] = np.sin(phi)
    return R

R = rotation(-np.pi/9)
print(np.linalg.det(R)) # should be one

plt.plot(v[:,0],v[:,1])
u = np.dot(R,v.T).T
plt.plot(u[:,0],u[:,1],"r-")
```


Affine transformations

Translation is not a linear transformation since it is not the same if we multiply the whole object by a factor of two then move it or the other way around, e.g.

```
def translate(u,t):
    return u+t

#translate vector
t = np.array([1,0])
# object multiplied by 2 and the translated:
v1 = translate(u*2,t)
v2 = translate(u,t) * 2
plt.plot(u[:,0],u[:,1])
plt.plot(v1[:,0],v1[:,1],"g-")
plt.plot(v2[:,0],v2[:,1],"r-")
```

Translation is affine transformation. The trick is that we add a new component to the input vector a simple 1 which after the matrix product will be added to the corresponding components:

$$\mathbf{u}' = \begin{pmatrix} \mathbf{u} \\ 1 \end{pmatrix}$$

The affine transformation is then

$$A' = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} t_x & t_y \end{pmatrix}$$

Let us do the multiplication:

$$v'_j = \sum_{i=1}^3 A'_{ji} u'_i = u_j + t_j.$$

Note that in the top part of A' we put the identity matrix, but any linear operation there is executed the translated.

```
def translation(t):
    A = np.zeros((2,3),dtype=float)
    A[0,0] = 1
    A[1,1] = 1
    A[:,2] = t
    return A

T = translation(np.array([1,-0.5]))
print(T)

def tdot(T,u):
    if u.ndim == 1:
        uu = np.ones(3,dtype=float)
        uu[2] = u
        return np.dot(T,uu)
    else:
        shape = list(u.shape)
```

```

        shape[0] += 1
        uu = np.ones(shape, dtype=float)
        uu[:-1] = u
        vT = np.dot(T, uu)
        return(vT.T)

tdot(T, u.T)

plt.plot(v[:,0], v[:,1])
u = tdot(T, v.T)
plt.plot(u[:,0], u[:,1], "r-")

```

As mentioned earlier We can you any linear transformation along with translation in the affine transformation.

```

def affine(B, t):
    A = np.zeros((2,3), dtype=float)
    A[:2,:2] = B
    A[:,2] = t
    return A

A = affine(rotation(np.pi/4), np.array([1, -0.5]))
print(A)

plt.plot(v[:,0], v[:,1])
u = tdot(A, v.T)
plt.plot(u[:,0], u[:,1], "r-")

```

Affine transformations in 3d

Now let us do it properly! First we need a visualization tool. We will use `plotly`

```

# plot_cube takes the corners of a parallelepiped, and plots it.
# Arrays can be supplied instead of the coordinates and the color(s)
for multiple objects
def plot_cube(X, Y, Z, color='blue'):
    if '[' in str(X[0]): # ] just close it :-)
        fig = go.Figure(data=[
            go.Mesh3d(
                # 8 vertices of a cube
                x=X[0],
                y=Y[0],
                z=Z[0],

                i = [0,3,0,5,0,6,4,7,2,7,1,7],
                j = [1,1,1,1,2,2,5,5,3,3,3,3],
                k = [2,2,4,4,4,4,6,6,6,6,5,5],
                opacity=0.6,
                color=color[0],

```

```

        flatshading = True
    )
])
for i in range(1,len(X)):
    fig.add_trace(go.Mesh3d(
        # 8 vertices of a cube
        x=X[i],
        y=Y[i],
        z=Z[i],

        i = [0,3,0,5,0,6,4,7,2,7,1,7],
        j = [1,1,1,1,2,2,5,5,3,3,3,3],
        k = [2,2,4,4,4,4,6,6,6,6,5,5],
        opacity=0.6,
        color=color[i],
        flatshading = True
    ))
else:
    fig = go.Figure(data=[
        go.Mesh3d(
            # 8 vertices of a cube
            x=X,
            y=Y,
            z=Z,

            i = [0,3,0,5,0,6,4,7,2,7,1,7],
            j = [1,1,1,1,2,2,5,5,3,3,3,3],
            k = [2,2,4,4,4,4,6,6,6,6,5,5],
            opacity=0.6,
            color=color,
            flatshading = True
        )
    ])
fig.show()

X = np.arange(8)//4
Y = (np.arange(8)//2)%2
Z = np.arange(8)%2
plot_cube(X,Y,Z)

#two cubes (We shift the second by two units in the x direction to be
able to see anything.)
plot_cube([X-1,X+1],[Y,Y],[Z,Z],color=['red','blue'])

```

Transformations

- Scaling
- Projection
- Rotation
- Translation

```

#Scaling
def L_scale(A,alpha):
    return (A.T*alpha).T

#projection
def L_projection(A,t):
    tt = t / np.linalg.norm(t)
    return np.dot(np.identity(len(A)) - np.outer(tt,tt),A)

#projection
def L_rotation_xyz(A,alpha):
    ax = alpha[0]
    ay = alpha[1]
    az = alpha[2]
    Rx = np.array([[1.0,0.0,0.0],[0,np.cos(ax),-np.sin(ax)],
    [0,np.sin(ax),np.cos(ax)]])
    Ry = np.array([[np.cos(ay),0,np.sin(ay)],[0.0,1.0,0.0],[-
    np.sin(ay),0,np.cos(ay)]])
    Rz = np.array([[np.cos(az),-np.sin(az),0],
    [np.sin(az),np.cos(az),0],[0.0,0.0,1.0]])
    return np.dot(Rz,np.dot(Ry,np.dot(Rx,A)))

#translation, which we implement in a simple way
def L_translate(A,t):
    return (A.T+t).T

#coordinates:
X = np.arange(8)//4
Y = (np.arange(8)//2)%2
Z = np.arange(8)%2
# in a single matrix
A = np.concatenate((X,Y,Z)).reshape(3,-1)

B = L_scale(A,np.array([0.5,2,1.3]))
plot_cube([X-1,B[0]+1],[Y,B[1]],[Z,B[2]],color=['red','blue'])

B = L_projection(A,np.array([0.5,2,1.3]))
plot_cube([X-1,B[0]+1],[Y,B[1]],[Z,B[2]],color=['red','blue'])

B = L_rotation_xyz(A,np.array([np.pi/6,np.pi/4,np.pi/12]))
plot_cube([X-1,B[0]+1],[Y,B[1]],[Z,B[2]],color=['red','blue'])

#Let us shift the rotated matrix back
C = L_rotation_xyz(A,np.array([np.pi/6,np.pi/4,np.pi/12]))
B = L_translate(C,np.array([-2,0,0]))
plot_cube([X-1,B[0]+1],[Y,B[1]],[Z,B[2]],color=['red','blue'])

```